

هندسة البرمجيات (Software Engineering)

الفصل السابع: الإختبار (Testing)

المخطط العام – Outline

- Testing
- Unit Testing
- JUnit 4.x
- JUnit in NetBeans
- Test Fixtures
- Assertions
- Acceptance Test

الاختبار – Testing

بشكل عام، اختبار البرمجيات يعني تشغيل البرنامج في ظروف مُتحكم بها من أجل:

1. التحقق من أن سلوك البرنامج "كما هو محدد".
2. اكتشاف الأخطاء.
3. التحقق من أن ما تم تحديده فعلاً هو ما يريده المستخدم.

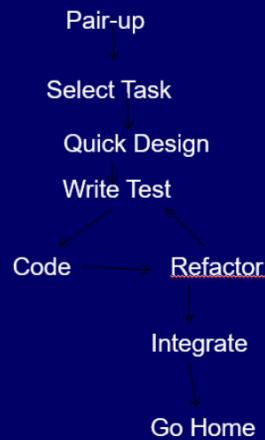
الاختبار في البرمجة القصوى (XP)

(Testing in XP)

- يتم إجراء الاختبار بشكل مستمر طوال عملية التطوير.
- يكتب المبرمجون الاختبارات أولاً، ثم يطورون البرنامج لتحقيق متطلبات هذه الاختبارات.
- يقوم العميل أيضًا بتوفير (Acceptance Tests-اختبارات القبول) في كل مرحلة لضمان الوصول إلى النتائج المطلوبة.

XP Process

An XP developer's daily routine



الاختبار المستمر (Continuous Testing)



- قبل إضافة أي ميزة، يقوم المبرمجون بكتابة اختبار لها.
- عند اجتياز مجموعة الاختبارات، يعتبر العمل قد اكتمل.
- **في XP، هناك نوعان أساسيان من الاختبارات:**

اختبارات الوحدة – Unit Tests



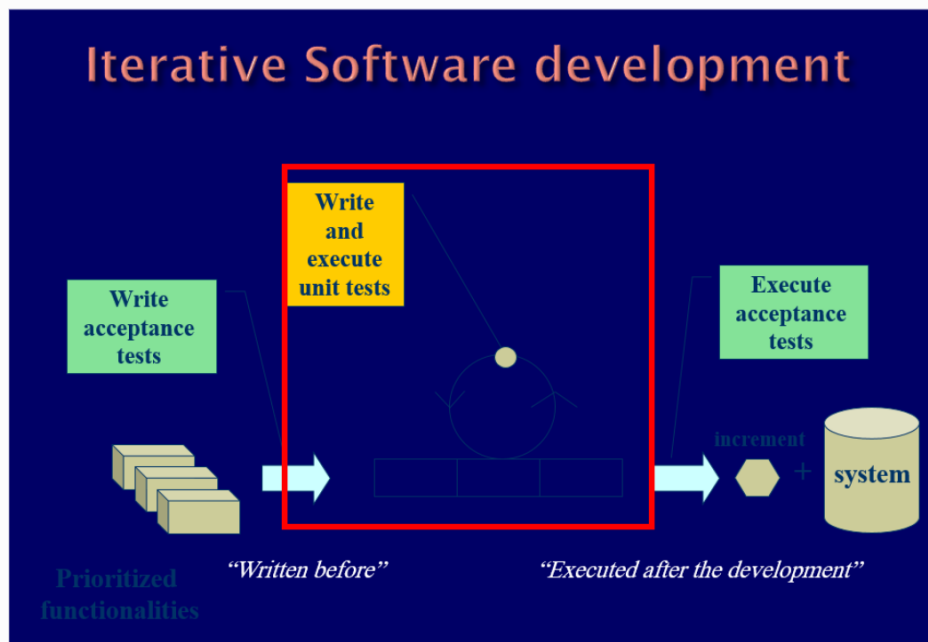
- اختبارات مؤتمتة (Automated Tests) يكتبها المطورون لاختبار الوظائف أثناء تطويرها.
- كل اختبار وحدة يختبر صفاً واحدًا (Class) أو مجموعة صغيرة من الصفوف.
- تُكتب باستخدام إطار اختبار وحدات مثل JUnit.

اختبارات القبول – Acceptance Tests



أو (Functional Tests)

- يحددها العميل لاختبار أن النظام ككل يعمل كما هو محدد.
- تختبر النظام بالكامل أو جزءًا كبيرًا منه.
- عند اجتياز جميع اختبارات القبول لقصة مستخدم معينة، تُعتبر هذه القصة مكتملة.
- يمكن أن تتكون من سكريبت (Script) لتفاعلات واجهة المستخدم (UI) والنتائج المتوقعة (تشغيل يدوي)، ولكن من الأفضل أن تكون مؤتمتة باستخدام إطار اختبار قبول خاص أو عبر JUnit.



الاختبار في XP (XP Testing)



- كل جزء من الشيفرة البرمجية يجب أن يحتوي على اختبارات وحدة مؤتمتة (Automated Unit Tests).
- يتم رفع الاختبارات مع الكود إلى المستودع.
- يكتب المبرمجون اختبارات الوحدة قبل كتابة الكود، ويضيفون اختبارات جديدة عند اكتشاف أي نقص.
- لا يُعتبر أي تعديل (Modification) أو (Refactoring) مكتملاً حتى تنجح جميع اختبارات الوحدة بنسبة 100%.
- ال Acceptance Testing Framework يُستخدم لإجراء اختبارات القبول المؤتمتة.
- تقوم إختبارات القبول (Acceptance Tests) باختبار كتل كبيرة من الوظائف مثل User Stories.
- تُعتبر القصة مكتملة عند اجتياز جميع اختبارات القبول الخاصة بها.

التطوير الموجه بالاختبار



(Test-Driven Development - TDD)

- ال Test First قبل إضافة ميزة، اكتب اختباراً لها!
- إذا لم يكن هناك اختبار مؤتمت، يُفترض أن الكود لا يعمل.
- عندما تجتاز مجموعة الاختبارات الكاملة بنسبة 100%، تُقبل الميزة.

اختبارات الوحدة – Unit Tests



- تؤتمت اختبارات الوظائف أثناء التطوير.
- تكتب باختبار صف واحد أو مجموعة صغيرة من الصفوف.
- تُستخدم أطر عمل مثل **JUnit** أو **xUnit**.
- التجارب أظهرت أن **TDD** يقلل من وقت تصحيح الأخطاء (Debugging).
- يعزز الثقة بأن الميزات الجديدة تعمل وتتكامل مع النظام.
- عند اكتشاف **Bug** أثناء التطوير، يجب كتابة اختبار له لضمان عدم عودته.

الاختبار باستخدام JUnit



(Testing with JUnit)

ال **JUnit** هو بيئة اختبار وحدة لبرامج Java، طُوِّر بواسطة

Erich Gamma & Kent Beck

يشمل العمل مع **JUnit**:

- كتابة حالات الاختبار (Test Cases).
- تنفيذ حالات الاختبار.
- التحقق من النجاح أو الفشل (هل النتيجة المتوقعة = النتيجة الفعلية؟)

كيفية كتابة الاختبارات



(How to Write Tests?)

- يجب أن تكون جميع الاختبارات من النوع **Pass/Fail** (نجاح / فشل).

✗ مخرجات اختبار سيئة:

Now Testing Person.java:

First Name: Tanner

Last Name: Burke

Age: 1

✓ مخرجات اختبار جيدة:

Now Testing Person.java:

Failure: Expected Age 2, but was 1 instead.

ال JUnit 4.x

ال JUnit هو إطار اختبار (Testing Framework) بسيط يوفر ما يلي:

- ال Annotations لتحديد أن الدوال هي اختبارات (@Test)
- ال Annotations لإعداد وتنظيف البيانات (تُعرف بـ **Fixtures**) **مثل:** `@Before`, `@After`, `@BeforeClass`, `@AfterClass`
- ال Methods لكتابة Assertions (الافتراضات)

بسيط JUnit

(Simple JUnit)

- أنشئ **Test Class & Test Case** (صف اختبار & حالة اختبار).
- استخدم دوال **assert** للتحقق من صحة ناتج الطريقة، **مثل:** `assertEquals()`, `assertTrue()`, `assertNotNull()`
- يمكن تشغيل الاختبار يدويًا بتشغيل الصف، أو بشكل آلي باستخدام **Ant Script**.

1. أنشئ Class بسيط

```
package Test;
public class MyClass {
    public int multiply(int x, int y) {
        return x / y; // خطأ متعمد للاختبار
    }
}
```

2. أنشئ Test Class

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;
public class MyClassTest {
    @Test
    public void testMultiply() {
        MyClass tester = new MyClass();
        assertEquals("Result", 50, tester.multiply(10, 5));
    }
}
```

3. شغل الاختبار

يجب أن يفشل الاختبار (يظهر شريط أحمر)، والسبب: 

java.lang.AssertionError: Result expected <50> but was: <2>

✗ لأن العملية داخل multiply تستخدم القسمة / بدل الضرب *

ال JUnit in NetBeans



- لا حاجة لإضافة مكتبة JAR يدويًا إلى المشروع.
- افتراضيًا، مكتبة JUnit JAR موجودة ضمن مجلد Test Library.
- يوجد في NetBeans قائمة لتوليد كود Test Class & Test Case تلقائيًا.

Let's Do The Code



لنبدأ البرمجة الخطوات:

- افتح NetBeans 6.1 أو أي إصدار أحدث.
- أنشئ مشروع Java جديد.
- أنشئ Class بسيط يحتوي على:
 - دالة ذات قيمة مرجعة (مثل الجمع أو الطرح).
 - دالة void (تطبع شيئًا على الشاشة فقط).

إنشاء اختبار وحدة (Create Unit Test)



- من داخل NetBeans، اختر: Tools > Create JUnit Test
- أو ببساطة اضغط على: Ctrl + Shift + U
- ستظهر نافذة حوارية، اختر الإعدادات المناسبة أو اتركها كما هي.
- سيتم إنشاء ملف Test Case تلقائيًا داخل مجلد test package.

اختبار الوحدة (Unit Testing)



- قم بإعطاء قيمة للمتغيرات داخل دالة الاختبار.
- احذف الدالة fail() إذا كانت موجودة،
 - (تُستخدم مؤقتًا للإشارة لاختبار غير مكتمل).
- شغل ملف الاختبار باستخدام: Shift + F6.
- راقب نتيجة الاختبار في نافذة النتائج.

```

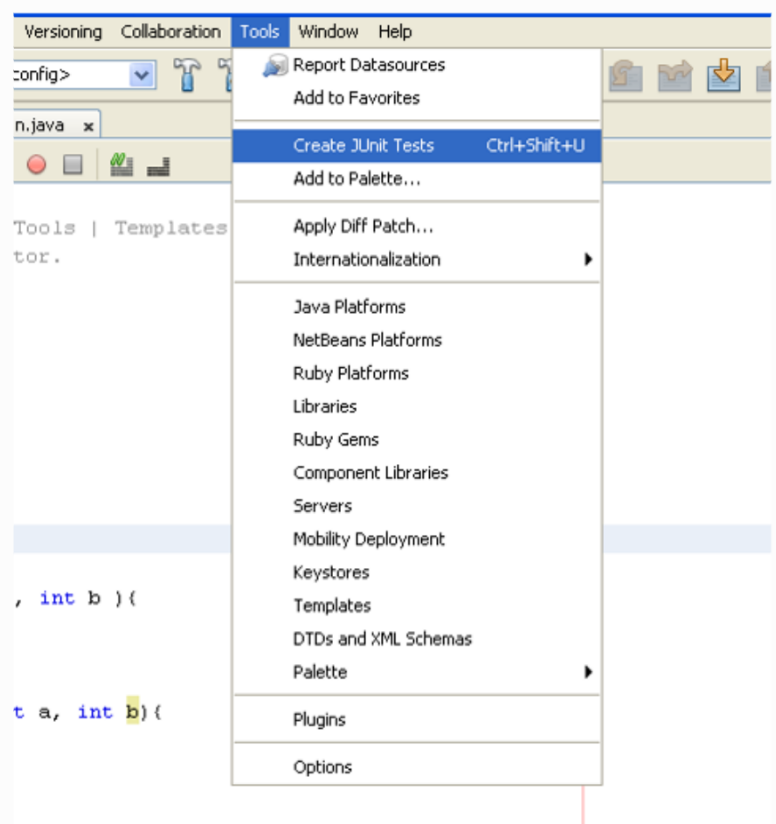
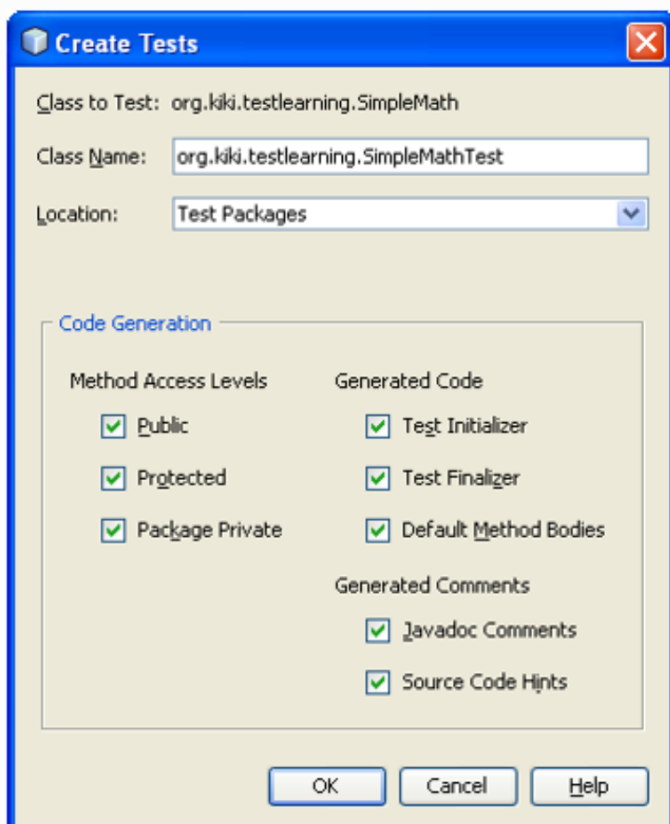
10  * @author Kiki
11  */
12  public class SimpleMath {
13
14
15      public int doAddition(int a, int b ){
16
17          return a + b ;
18      }
19      public int doSubtraction(int a, int b ){
20
21          return a / b;
22      }
23
24      public void printAddition(int a, int b){
25
26          System.out.println("var1 = "+a+" , var2 = "+b+" " +
27              "hasilnya adalah = "+doAddition(a, b));
28
29      }
30  }

```

```

11  */
12  public class SimpleMath {
13
14
15      public int doAddition(int a, int b ){
16
17          return a + b ;
18      }
19      public int doSubtraction(int a, int b ){
20
21          return a / b;
22      }
23
24      public void printAddition(int a, int b){
25
26          System.out.println("var1 = "+a+" , var2 = "+b+" " +
27              "hasilnya adalah = "+doAddition(a, b));
28
29      }
30
31
32
33  }

```



```

41 |         * Test of doAddition method, of class SimpleMath.
42 |         */
43 |     @Test
44 |     public void testDoAddition() {
45 |         System.out.println("doAddition");
46 |         int a = 2;
47 |         int b = 2;
48 |         SimpleMath instance = new SimpleMath();
49 |         int expResult = 4;
50 |         int result = instance.doAddition(a, b);
51 |         assertEquals(expResult, result);
52 |         // TODO review the generated test code and remove the default call to fail.
53 |
54 |     }
55 |
56 |     /**
57 |      * Test of doSubtraction method, of class SimpleMath.
58 |      */
59 |     @Test
60 |     public void testDoSubtraction() {
61 |         System.out.println("doSubtraction");
62 |         int a = 3;
63 |         int b = 1;
64 |         SimpleMath instance = new SimpleMath();
65 |         int expResult = 3;
66 |         int result = instance.doSubtraction(a, b);
67 |         assertEquals(expResult, result);
68 |         // TODO review the generated test code and remove the default call to fail.
69 |
70 |     }

```

JUnit Test Results

All 3 tests passed.

- org.kiki.testlearning.SimpleMathTest passed
 - testDoAddition passed (0.015 s)
 - testDoSubtraction passed (0.0 s)
 - testPrintAddition passed (0.0 s)

```

doAddition
doSubtraction
printAddition
var1 = 3 , var2 = 3 hasilnya adalah
|

```

HTTP Monitor Output JUnit Test Results



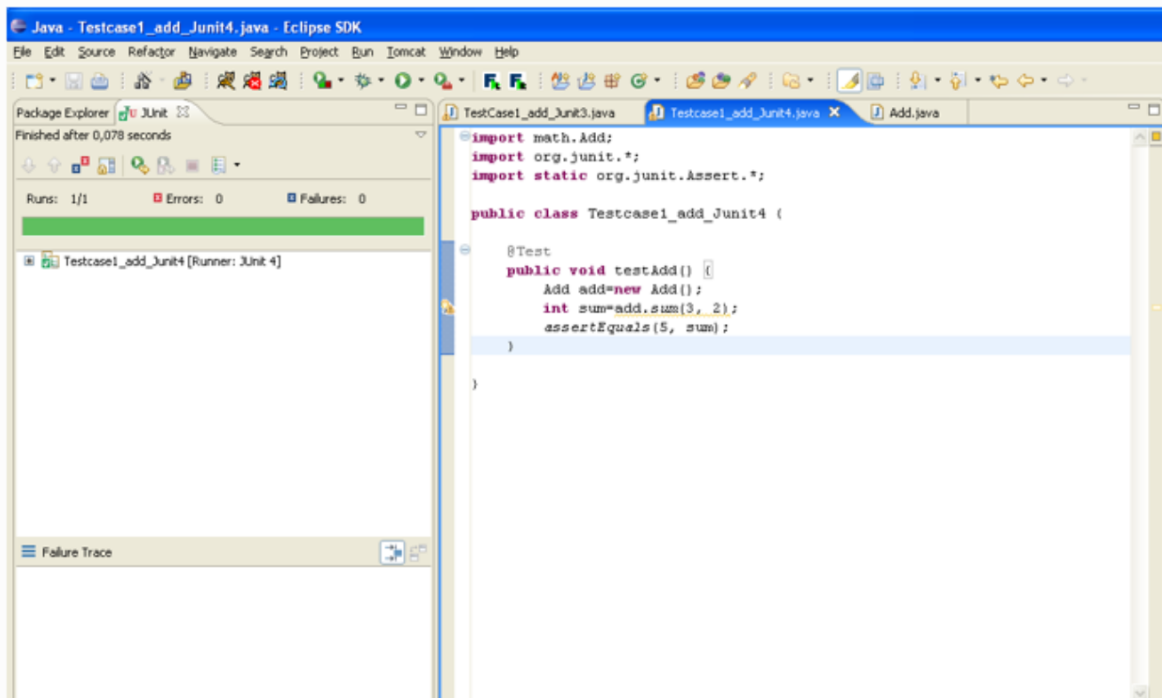
Another Example (JUnit 4.x)

✓ مثال صحيح (يجب أن ينجح):

```
// Class to be tested
public class Add {
    public static int sum(int a, int b) {
        return a + b;
    }
}

// Test class
import math.Add;
import org.junit.*;
import static org.junit.Assert.*;

public class Testcase1_add_Junit4 {
    @Test
    public void testAdd() {
        Add add = new Add();
        int sum = add.sum(3, 2);
        assertEquals(5, sum);
    }
}
```



✖ مثال به خطأ في الكود (الاختبار سيفشل):

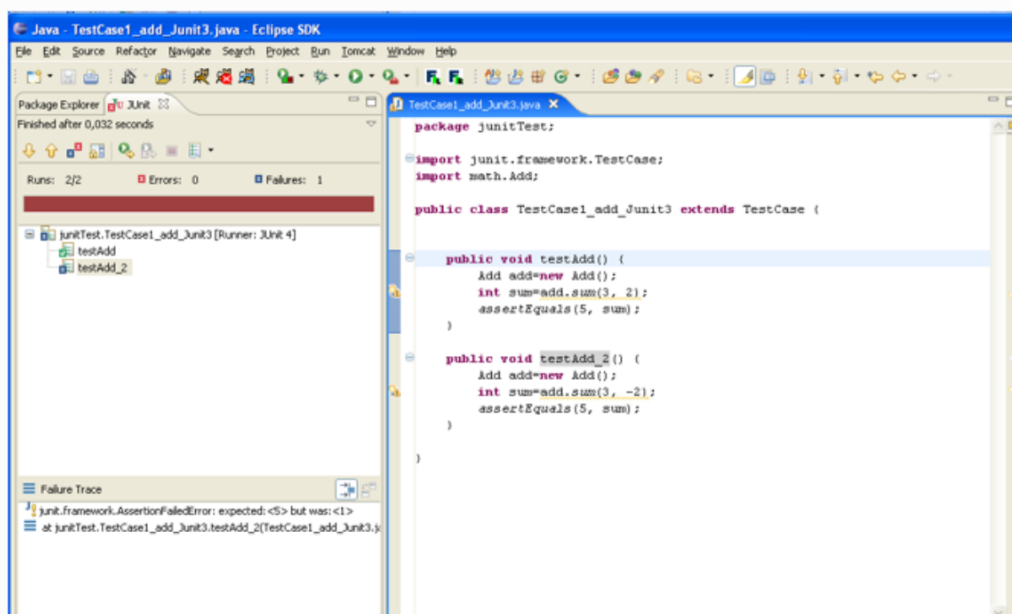
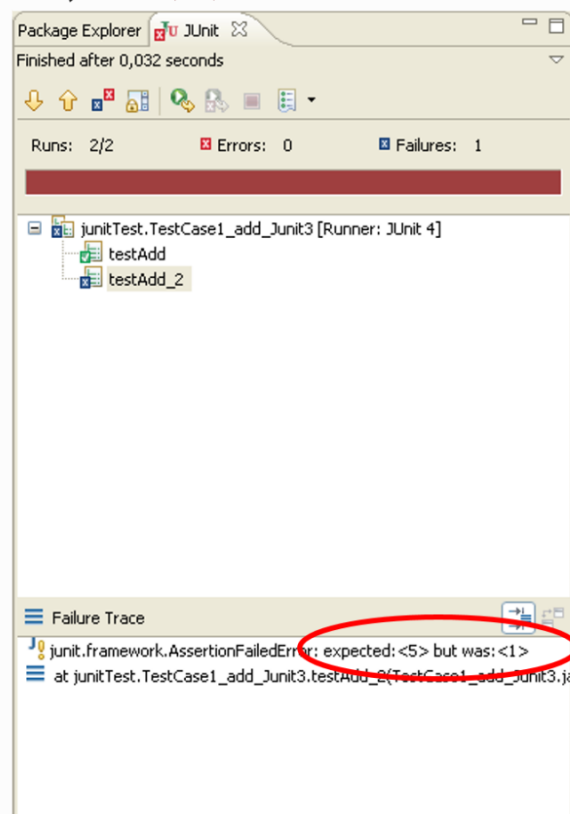
```
// Class to be tested
public class Add {
    public static int sum(int a, int b) {
        return a - b; // خطأ متعمد ✖
    }
}

// Test class
import math.Add;
import org.junit.*;
import static org.junit.Assert.*;
public class Testcase1_add_Junit4 {
    @Test
    public void testAdd() {
        Add add = new Add();
        int sum = add.sum(3, 2);
        assertEquals(5, sum); // ✖ Will Fail (Expected 5, got 1)
    }
}
```




Example Using JUnit 3.x

```
import junit.framework.TestCase;
import math.Add;
public class TestCase1_add_Junit3 extends TestCase {
    public void testAdd() {...}
    public void testAdd_2() {
        Add add = new Add();
        int sum = add.sum(3, -2);
        assertEquals(5, sum); // ❌ sum = a - b إذا كانت الدالة
```



VS Example: Old Way vs. New Way

(الطريقة التقليدية مقابل الحديثة)

```
int max(int a, int b) {
    if (a > b) { return a; }
    else { return b; }
}

void testMax() {
    int x = max(3, 7);
    if (x != 7) {
        System.out.println("max(3, 7) gives " + x);
    }
    x = max(3, -7); if (x != 3) {
        System.out.println("max(3, -7) gives " + x);
    }
}

public static void main(String[] args) {
    new MyClass().testMax(); // تنفيذ يدوي
}

@Test
public void testMax() {
    assertEquals(7, max(3, 7));
    assertEquals(3, max(3, -7));
}
```

تجهيزات الاختبار – Test Fixtures

Test Fixture هو السياق (Context) الذي يتم فيه تنفيذ Test Case.

عادةً ما تشمل تجهيزات الاختبار ما يلي:

- كائنات أو موارد (مثل ملفات أو قواعد بيانات) يمكن لأي اختبار استخدامها.
- الأنشطة اللازمة لتجهيز هذه الموارد (إنشاء، تخصيص، تهيئة...).
- تحرير أو حذف الموارد بعد انتهاء الاختبار.



JUnit في Fixtures الخاصة بـ Annotations

◆ @Before

تنفيذ دالة واحدة أو أكثر قبل كل اختبار: ✓

- تستخدم عادةً لتهيئة المتغيرات أو الموارد بحيث يبدأ كل اختبار من نقطة نظيفة.

```
@Before public void createOutputFile() {  
    output = new File(...); // مثال على تهيئة مورد  
}
```

◆ @After

تنفيذ دالة واحدة أو أكثر بعد كل اختبار: ✓

- تُستخدم لتحرير أو حذف الموارد التي تم إنشاؤها أثناء الاختبار.

```
@After public void deleteOutputFile() {  
    output.delete(); // حذف الملف بعد انتهاء الاختبار  
}
```

◆ @BeforeClass

تنفذ مرة واحدة فقط قبل جميع الاختبارات: ✓

- تُستخدم للتهيئة المكلفة مثل الاتصال بقاعدة البيانات أو تحميل ملفات ثقيلة.

```
@BeforeClass public static void setUpClass() throws Exception { // تهيئة  
    // لمرة واحدة فقط  
}
```

◆ @AfterClass

تنفذ مرة واحدة فقط بعد انتهاء جميع الاختبارات: ✓

- تُستخدم لأعمال التنظيف العامة مثل قطع الاتصال بقواعد البيانات أو حذف الملفات المؤقتة.

```
@AfterClass public static void tearDownClass() throws Exception { //  
    تنظيف لمرة واحدة فقط  
}
```

مثال عملي كامل ✓

```
import org.junit.*;
import static org.junit.Assert.*;
import java.util.*;
public class JunitTest1 {
    private Collection<String> collection;
    @BeforeClass
    public static void oneTimeSetUp() {
        System.out.println("@BeforeClass - oneTimeSetUp");
    }
    @AfterClass
    public static void oneTimeTearDown() {
        System.out.println("@AfterClass - oneTimeTearDown");
    }
    @Before
    public void setUp() { collection = new ArrayList<>();
        System.out.println("@Before - setUp");
    }
    @After
    public void tearDown() { collection.clear();
        System.out.println("@After - tearDown");
    }
    @Test
    public void testEmptyCollection() { assertTrue(collection.isEmpty());
        System.out.println("@Test - testEmptyCollection");
    }
    @Test
    public void testOneItemCollection() { collection.add("itemA");
        assertEquals(1, collection.size());
        System.out.println("@Test - testOneItemCollection");
    }
}
```

نتائج تنفيذ المثال: ✓

```
@BeforeClass - oneTimeSetUp
@Before - setUp
@Test - testEmptyCollection
@After - tearDown
@Before - setUp
@Test - testOneItemCollection
@After - tearDown
@AfterClass - oneTimeTearDown
```

ما هي Assertions؟

ال **Assertions** هي أدوات تحقق تُستخدم داخل اختبارات JUnit لضمان أن مخرجات الكود تتطابق مع التوقعات.

السلوك العام:

- إذا كانت ال **Assertion** صحيحة → يتم مواصلة تنفيذ الاختبار.
- إذا كانت ال **Assertion** خاطئة → يتوقف التنفيذ مباشرة، ويتم تسجيل الاختبار ك فشل (Fail).
- إذا حدث استثناء **Exception** → يتم تسجيل الاختبار ك خطأ (Error).
- إذا لم تفشل أي **Assertion** → يتم تسجيل الاختبار ك ناجح (Pass).

ملاحظات هامة:

- جميع دوال التحقق (Assertions) هي **Static** من الكلاس `org.junit.Assert`.
- كل دالة اختبار: يجب أن تكون معنونة بـ `@Test`.
- لا تأخذ أي معاملات.
- لا تُرجع قيمة.

مثال أساسي:

@Test

```
public void sumTest() {  
    assertEquals(15, program.sum(array)); // تحقق من المجموع  
    assertTrue(program.min(array) > 0); // تحقق أن أقل قيمة موجبة  
}
```

أنواع Assertions مع شرح مختصر:

الشروط البوليانية (Boolean):

القيم البوليانية تكون إما **true** (صحيحة) أو **false** (خاطئة):

- `assertTrue(condition)`

✓ يتحقق من أن الشرط صحيح (**true**).

- `assertFalse(condition)`

✓ يتحقق من أن الشرط خاطئ (**false**).

الكائنات (Objects) هل هي **null** أو غير **null**:

- `assertNull(object)`

✓ يتحقق من أن الكائن هو **null**.

- `assertNotNull(object)`

✓ يتحقق من أن الكائن ليس **null**.

الكائنات هل هي متطابقة أو غير متطابقة (أي نفس المرجع):

- `assertSame(expected, actual)`

صحيح فقط إذا: `expected == actual`

(أي يشيران إلى نفس الكائن في الذاكرة).

- `assertNotSame(expected, actual)`

يتحقق من أن `expected` و `actual`

(لا يشيران إلى نفس الكائن).

"المساواة" بين الكائنات (وفقًا لطريقة **equals**):

- `assertEquals(expected, actual)`

صحيح فقط إذا: `expected.equals(actual)`

(أي أن الكائنين متساويان من حيث القيمة وليس المرجع فقط).

📌 "المساواة" بين المصفوفات (Arrays):

• `assertArrayEquals(expected, actual)`

يجب أن تكون المصفوفات بنفس الطول.

• لكل قيمة صحيحة للفهرس i ، يتم التحقق من الآتي:

• `assertEquals(expected[i], actual[i])`

أو

• `assertArrayEquals(expected[i], actual[i])`

📌 مقارنة أنواع الأعداد العشرية (float أو double):

عند مقارنة أنواع البيانات العشرية، يجب تمرير معامل إضافي يسمى **delta**.

• يقوم الاختبار بالتحقق من المعادلة:

$\text{Math.abs(expected - actual)} \leq \text{delta}$

• هذا لمنع الأخطاء الناتجة عن تقريب الأعداد العشرية أثناء المقارنة.

✓ **مثال:** `assertEquals(aDouble, anotherDouble, 0.0001)`

🔧 متى نستخدم JUnit؟

• مخصص لاختبار الوحدات **Unit Testing**، أي اختبار أجزاء

صغيرة من الكود (مثل دوال أو كائنات).

• ليس مخصصًا لاختبار الأنظمة أو الأداء أو تكامل الأنظمة.

👤 **JUnit & Test-Driven Development (TDD) منهجية**:

• اكتب اختبار JUnit قبل كتابة الكود.

• اكتب أقل كمية من الكود تجعل الاختبار ينجح - فقط!

• شغل الاختبار → يجب أن ينجح.

• في كل مرة تضيف كود جديد → أعد تشغيل جميع الاختبارات لتتأكد أن لا شيء قد تعطل.

اختبار القبول – Acceptance Test

اختبارات القبول يتم إنشاؤها استنادًا إلى **user stories** (قصص المستخدم).

خلال كل **iteration** (تكرار تطويري)، يتم تحويل قصص المستخدم التي تم اختيارها خلال اجتماع التخطيط إلى اختبارات قبول.



اختبارات القبول في البرمجة المتطرفة (Acceptance Tests in XP)

النسخة البسيطة:

- العميل يكتب اختبارات القبول بمساعدة المطور و **user stories**.
- المطورون يكتبون الكود لجعل اختبارات القبول تنجح، ثم يقدمون النتائج للعميل.
- العميل يحدد السيناريوهات (**Scenarios**) التي يجب اختبارها للتحقق من أن قصة المستخدم قد تم تنفيذها بالشكل الصحيح.
- يمكن أن تحتوي القصة الواحدة على اختبار قبول واحد أو أكثر حسب الحاجة.

User Story: "Allow customer to make a deposit"

اختبار القبول: ✨

- Establish account (إنشاء حساب)
- Make deposit (إجراء إيداع)
- Verify balance is correct (التحقق من صحة الرصيد)

مثال أكثر تحديدًا: ✓

- Establish account
- Account number = 1231234556
- Account owner = John Smith
- Starting balance = 921.53
- Make deposit
- Deposit amount = 51.21
- Verify balance is correct
- Updated balance = 972.74

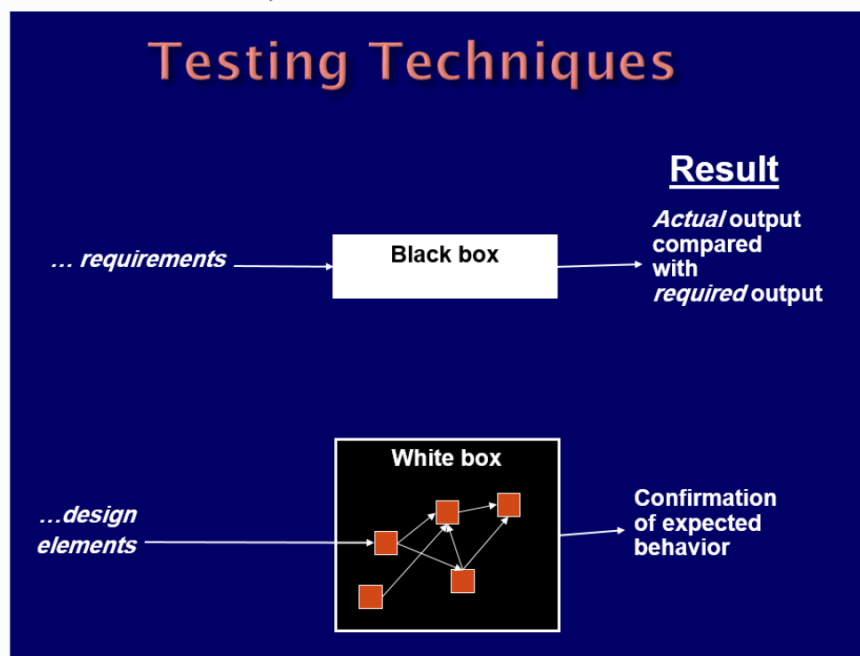


Acceptance Test = Black Box Test

اختبارات القبول تُعتبر black box system tests

(اختبارات نظام من نوع الصندوق الأسود).

كل اختبار يمثل نتيجة متوقعة من النظام.



اختبار الصندوق الأبيض (White Box Testing) 🔍

يعتمد على الخصائص الهيكلية والمنطقية الداخلية للبرنامج لاشتقاق بيانات الاختبار والتحقق.

اختبار الصندوق الأسود (Black Box Testing) 🧱

لا يعتمد على معرفة تفاصيل الكود الداخلي.
حالات الاختبار تُصمَّم بناءً على المواصفات فقط.

✓ مثال:

وظيفة البحث في مصفوفة عن قيمة:

- ال *Postcondition*: إرجاع مؤشر لحدوث القيمة، أو -1 إذا لم تكن موجودة.

أدوار العملاء في اختبار القبول 👤

- يتحمل العملاء مسؤولية التحقق من صحة اختبارات القبول.
- يراجعون نتائج الاختبار لتحديد الأولويات بين الاختبارات التي فشلت.

قبول القصة = اجتياز اختبارات القبول ↺

- قصة المستخدم لا تُعتبر مكتملة إلا إذا اجتازت جميع اختبارات القبول الخاصة بها.
- يجب إنشاء اختبارات قبول جديدة لكل iteration، وإلا فسيتم اعتبار التقدم = صفر.

✓ مثال تطبيقي لاختبار القبول – برنامج الكاشير

إنشاء فاتورة تسوق: Acceptance Test 📌

- مكونات السلة:

1 lb. coffee ☕

3 bags of cough drops 💊

1 gallon milk 🥛

- الأسعار:

Coffee = \$6/lb

Cough drops = \$2.49/bag

Milk = \$4.95/gallon

تحقق أن الإجمالي = 18.42\$ ✓

✓ مثال اختبار برمجي (كود):

```
inventory.setPrice("milk", 4.95);  
inventory.setPrice("cough drops", 2.49);  
inventory.setPrice("coffee", 6.00);  
order.addItem("milk", 1);  
order.addItem("cough drops", 3);  
order.addItem("coffee", 1);  
order.calculateSubtotal();  
assertEquals(order.receipt.getSubtotal(), 18.42);
```

✓ مثال اختبار يدوي عبر GUI:

- اختر milk من القائمة المنسدلة
- أدخل 1، ثم انقر "add"
- اختر coffee، أدخل 1، انقر "add"
- اختر cough drops، أدخل 3، انقر "add"
- تحقق أن Subtotal المعروض هو 18.42\$

⌚ ملاحظة:

- مفيدة للتنفيذ اليدوي، ولكن لا يُنصح بالاعتماد الكامل على هذه الطريقة نظرًا لبطئها وعدم فعاليتها في الاختبار التكراري-regression testing.

🎯 فوائد Acceptance Tests

- تُعد بمثابة عقد بين العميل والمطورين
- تفرض أن تكون القصص قابلة للاختبار
- تمكن العميل من تتبع التقدم بعدد الاختبارات ونجاحها
- تمنح المطورين ثقة بأن العمل يسير حسب الخطة
- تُعتبر القصة مكتملة فقط بعد اجتياز اختبارات القبول

⚠️ الفرق بين Acceptance Test و Unit Test

✗ ليسا متماثلين.

✓ من الممكن أن تنجح اختبارات الوحدة (Unit Tests) وتفشل اختبارات القبول.

💡 خاصة إذا كانت اختبارات القبول تتطلب تكامل بين مكونات عديدة

تم اختبارها فرديًا في Unit Tests.